UTFPR-CP

Professora: Gisele Alves Santana
Alunos: Ana Cecília Santana de Oliveira, Emily Vitorya de Moura e Andre Luiz Gonçalves da Silva Teixeira.

Pesquisa sobre o princípio SOLID - “Inversão de Dependência”:

● Definição

A *inversão de dependência* é a estratégia de depender de interfaces ou classes abstratas, ao invés de classes concretas. O DIP são módulos de alto nível, que não devem depender de módulos de baixo nível e ambos devem depender de abstrações, e abstrações não devem depender de detalhes, mas detalhes devem depender de abstrações.

● Objetivo

O objetivo principal do DIP é reduzir o acoplamento entre os diferentes módulos de um software, tornando o sistema muito mais fácil de manter, testar e evoluir ao longo do tempo.

● Aplicações

O DIP é aplicado sempre que precisamos isolar a lógica de negócio de detalhes técnicos que mudam com frequência ou que dependem de fatores externos. Como em Gateways de pagamento, em que o sistema precisa processar pagamentos via cartão de crédito, sem o DIP a classe de Pedido criaria diretamente uma instância da API do Stripe, e se a empresa decidir mudar para o PayPal, seria necessário reescrever a lógica. Com o DIP cria-se uma interface chamada ProcessadorDePagamento, a classe Pedido depende apenas dessa interface e o Stripe e o PayPal viram apenas detalhes de implementação que podem ser trocados sem alterar uma única linha da lógica do pedido.

● Exemplo

Aqui, a classe CadastroUsuario está fortemente acoplada à implementação concreta do SendGridEmailService:

```
// Módulo de Baixo Nível (Detalhe técnico)
class SendGridEmailService {
    public void enviarEmail(String para, String mensagem) {
        System.out.println("Enviando e-mail via SendGrid para " + para + ": " +
mensagem);
    }
}
```

```
// Módulo de Alto Nível (Regra de negócio)
class CadastroUsuario {
    // Dependência direta de uma classe concreta (Violando o DIP)
    private SendGridEmailService emailService;

    public CadastroUsuario() {
        // Instanciação direta cria um acoplamento rígido
        this.emailService = new SendGridEmailService();
    }

    public void executar(String email) {
        System.out.println("Usuário salvo no banco de dados.");
        this.emailService.enviarEmail(email, "Bem-vindo ao sistema!");
    }
}
```

Com o DIP, para corrigir isso, criamos uma interface que define o contrato de notificação:

```
// 1. A Abstração (O Contrato)
interface Notificador {
    void enviar(String para, String mensagem);
}

// 2. Módulos de Baixo Nível (Implementações do contrato)
class SendGridEmailService implements Notificador {
    @Override
    public void enviar(String para, String mensagem) {
        System.out.println("[E-mail SendGrid] Enviando para " + para + ": " +
mensagem);
    }
}

class WhatsAppService implements Notificador {
    @Override
    public void enviar(String para, String mensagem) {
        System.out.println("[WhatsApp] Enviando para " + para + ": " + mensagem);
    }
}

// 3. Módulo de Alto Nível (Regra de negócio independente)
class CadastroUsuario {
```

```
    // Depende apenas da interface (Respeitando o DIP)
    private final Notificador notificador;

    // A dependência é injetada via construtor
    public CadastroUsuario(Notificador notificador) {
        this.notificador = notificador;
    }

    public void executar(String contato) {
        System.out.println("Usuário salvo no banco de dados.");
        this.notificador.enviar(contato, "Bem-vindo ao sistema!");
    }
}

// === EXECUÇÃO DO CÓDIGO (MAIN) ===
public class Main {
    public static void main(String[] args) {
        // Se o sistema decidir usar E-mail:
        Notificador servicoEmail = new SendGridEmailService();
        CadastroUsuario cadastroComEmail = new CadastroUsuario(servicoEmail);
        cadastroComEmail.executar("usuario@email.com");

        System.out.println("-----------------------------------");

        // Se o sistema decidir mudar para WhatsApp, a classe CadastroUsuario
permanece intacta:
        Notificador servicoWhats = new WhatsAppService();
        CadastroUsuario cadastroComWhats = new CadastroUsuario(servicoWhats);
        cadastroComWhats.executar("+5511999999999");
    }
}
```